



Setting up Typescript + ESLint + Jest Project

Vishal Sharma · [Follow](#)

6 min read · Jul 7, 2021



Listen



Share

A lot of node projects have a setup that usually includes Typescript for static type checking, eslint for static code analysis, and Jest as a test runner. In this article, we will go through some advanced configurations for these three and how one can be leveraged from the other. So let's get started:

• • •

Step 1: Setup a basic node project

Let's start by creating a simple node project.

```
mkdir ts-eslint-jest-setup  
yarn init --yes
```

Once the above steps are done, the `package.json` in the project will look something similar to:



Step 2: Install the required dependencies

Let's start adding the most basic dependencies we need for our setup

```
yarn add -D typescript eslint jest
```

Once installed, add the basic script tasks in package.json to execute the commands.

```
package.json

{
  "name": "ts-eslint-jest-setup",
  "version": "1.0.0",
  "main": "index.js",
  "license": "MIT",
  "scripts": {
    "build": "tsc",
    "dev": "yarn build --watch --preserveWatchOutput",
    "lint": "eslint src --ext js,ts",
    "test": "jest"
  },
  "dependencies": {},
  "devDependencies": {
    "eslint": "^7.27.0",
    "jest": "^26.6.3",
    "typescript": "^4.2.4"
  }
}
```

Step 3: Set up Typescript

Run the command to initialize the typescript in the project.

```
yarn tsc --init
```

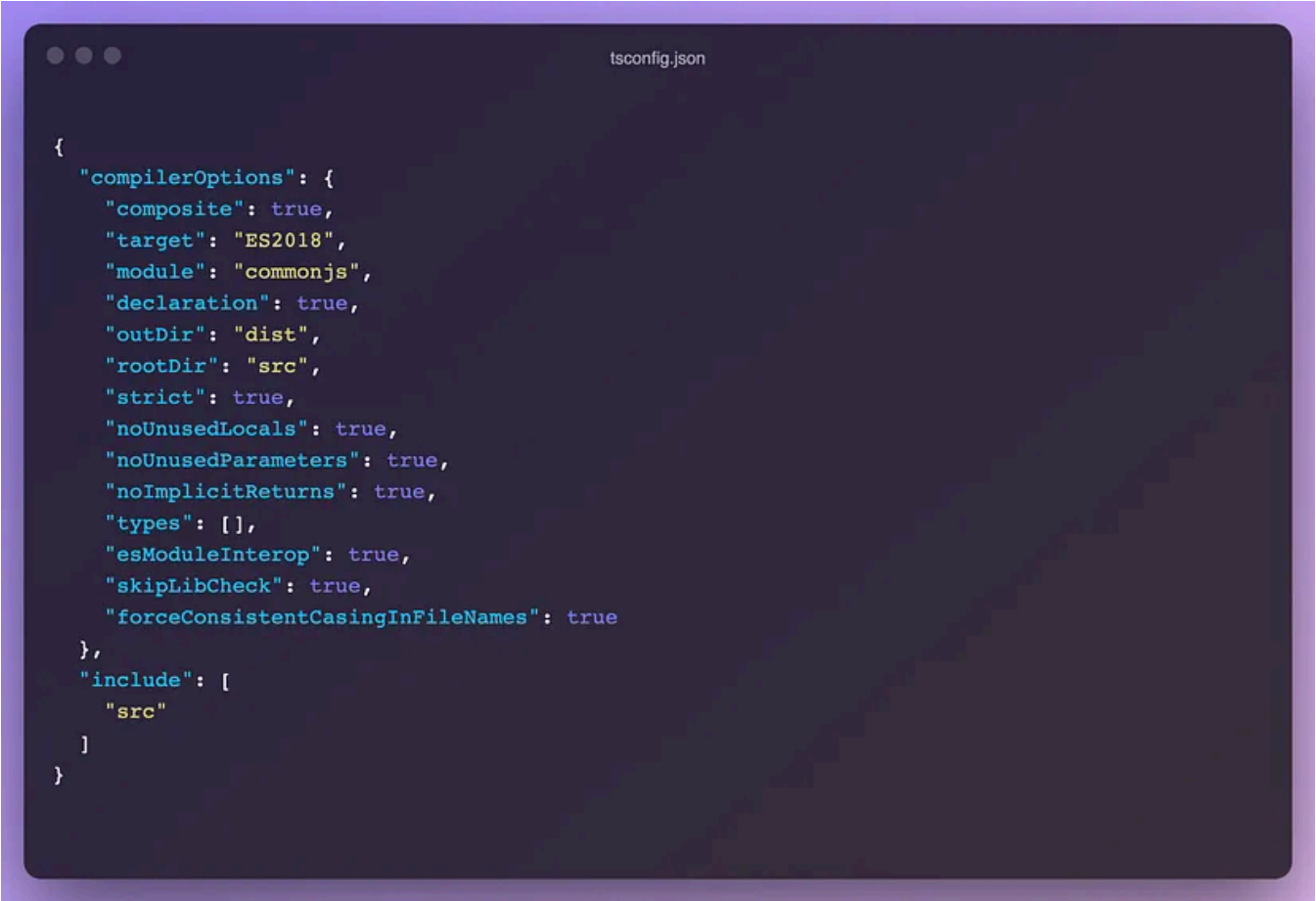
By default, the typescript compiler adds the configuration file `tsconfig.json` in the project root directory. we need to change a few settings in the file for our setup to work as expected.

- **target:** This property is the language level we wish to support in the project. It means what are we compiling out to? Changing this to `ES2018` would mean that the tsc output will be at ECMA 2018 level.
- **outDir:** By default, the typescript compiler will output in the src folder itself. But we want the output to get generated into a separate folder altogether so that if

we need to remove the output, we can easily do it by `rm -rf dist` changing the `outDir` option to `"outDir": "dist"` will achieve that for us. With this `src` and `dist` will be two separate folders.

- **rootDir:** changing `outDir` options will help us to create a separate folder, but it will still have a `src` folder inside it. We want the `dist` folder to have the output files directly inside the `dist` folder instead of inside the `src`. Changing `"rootDir": "src"` will achieve that for us.
- **include:** add `"include": ["src"]` outside the compiler options to tell the compiler to include all files inside the `src` folder.
- **declaration:** set `"declaration": true` to generate the type information `d.ts` files.
- **types:** By default, anything found in `node_modules/@types` is available for auto import across the app. set `"types": []` so that not all global scoped types are available in the `src` folder. e.g. you don't want `jest` types to be available in `src` files.

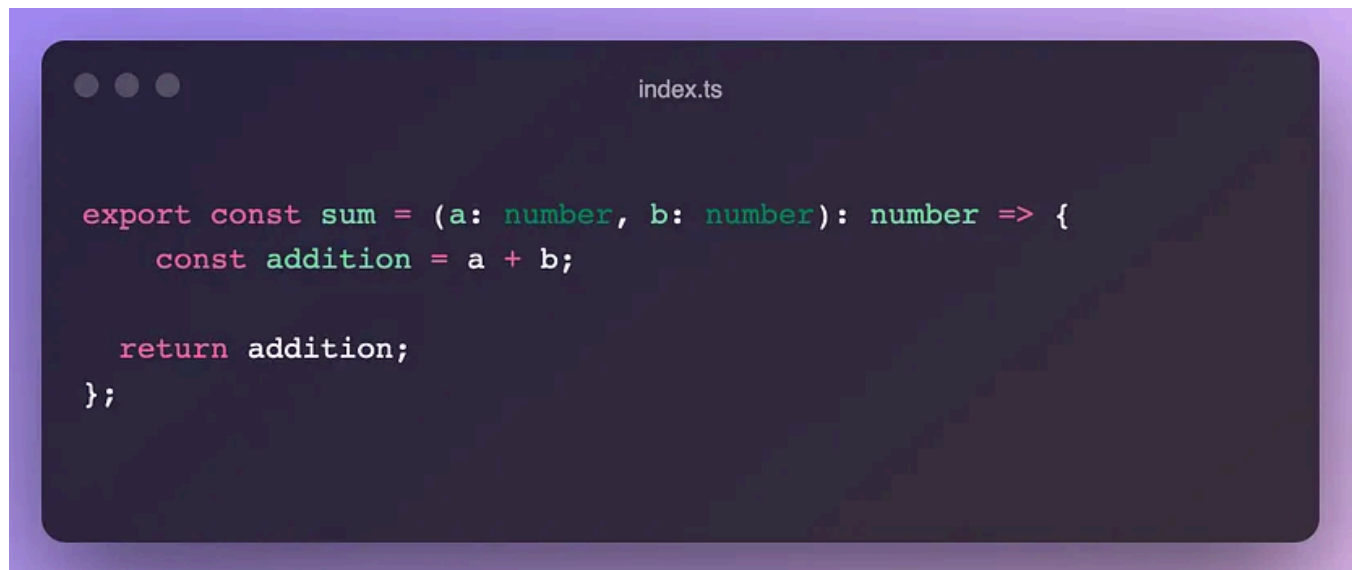
After setting up the above options, `tsconfig.json` file will look similar to:



```
{
  "compilerOptions": {
    "composite": true,
    "target": "ES2018",
    "module": "commonjs",
    "declaration": true,
    "outDir": "dist",
    "rootDir": "src",
    "strict": true,
    "noUnusedLocals": true,
    "noUnusedParameters": true,
    "noImplicitReturns": true,
    "types": [],
    "esModuleInterop": true,
    "skipLibCheck": true,
    "forceConsistentCasingInFileNames": true
  },
  "include": [
    "src"
  ]
}
```

Now, this is the time to add a code in the src folder and test the output.

Create a `index.js` file and add a basic function to add two numbers.

A screenshot of a code editor window with a dark background and light-colored text. The window title is 'index.ts'. The code inside is a TypeScript function definition for a sum function. The code is as follows:

```
export const sum = (a: number, b: number): number => {  
  const addition = a + b;  
  
  return addition;  
};
```

Now, if you run `npm build`, a `dist` folder will be generated with `index.ts` and `index.d.ts` files.

At last, if you are building a library, set `"main": "dist/index.ts"` and `"types: index.d.ts"` in the `package.json` files to tell the consumer what is the starting point of your library.

Step 4: Set up ESLint

ESLint is used for static code analysis. It has some great functionality out of the box but if your project support react, typescript, or vue, you would need to add more plugins.

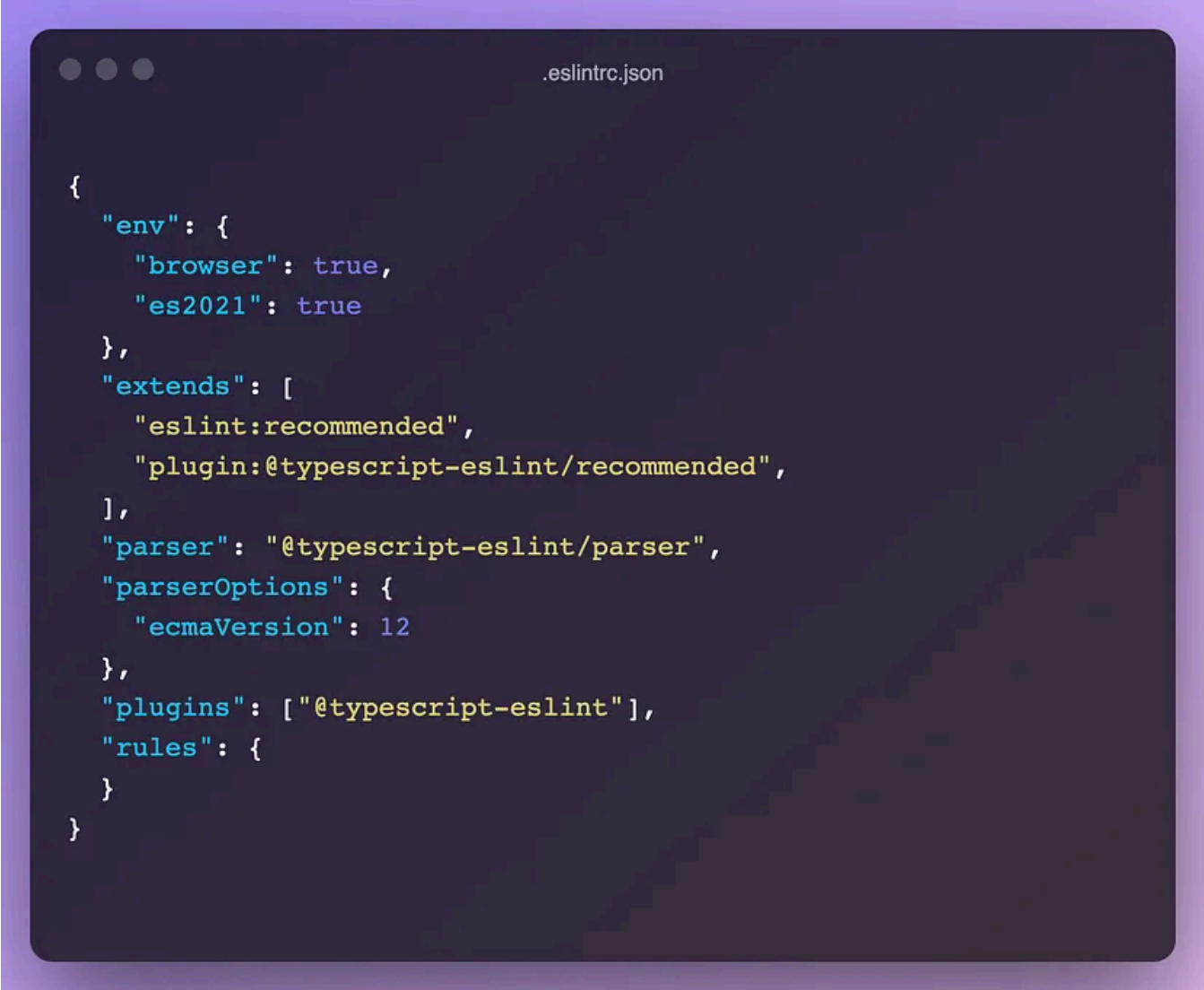
To initialize the eslint, run the following code:

```
yarn eslint --init
```

it will ask a bunch of questions, and you can setup according to your project needs. Since we are setting up ESLint with typescript, make sure the option is enabled while initializing. Once the initialization is done, it will add a few typescript related ESLint plugins,

```
@typescript-eslint/eslint-plugin@latest
@typescript-eslint/parser@latest
```

the `.eslintrc.json` will look similar to:

A screenshot of a code editor window with a dark background and light-colored text. The window title is ".eslintrc.json". The code is a JSON object with the following structure: an "env" object with "browser" and "es2021" set to true; an "extends" array containing "eslint:recommended" and "@typescript-eslint/recommended"; a "parser" set to "@typescript-eslint/parser"; a "parserOptions" object with "ecmaVersion" set to 12; a "plugins" array containing "@typescript-eslint"; and a "rules" object which is currently empty. The code is formatted with syntax highlighting: strings are in quotes, booleans are in blue, and array elements are separated by commas.

```
{
  "env": {
    "browser": true,
    "es2021": true
  },
  "extends": [
    "eslint:recommended",
    "plugin:@typescript-eslint/recommended",
  ],
  "parser": "@typescript-eslint/parser",
  "parserOptions": {
    "ecmaVersion": 12
  },
  "plugins": ["@typescript-eslint"],
  "rules": {
  }
}
```

ESLint recommended is a very basic set of typescript rules added by default.

`@typescript-eslint/recommended` adds rules specific to typescript that are very cheap to evaluate. You can opt-in additional typescript checks by adding `@typescript-eslint/recommended-requiring-type-checking` which require more type info and are costly to evaluate also. We can add rules specific to our project needs inside the `rules` object.

If we try to run `yarn lint` now, it will throw an error:

```
Oops! Something went wrong! :(
ESLint: 7.29.0
Error: Error while loading rule '@typescript-eslint/await-thenable': You have used a rule which requires parserServices to be generated. You must therefore provide a value for the "parserOptions.project" property for @typescript-eslint/parser.
Occurred while linting /Users/vishalsh/workspace/my-project/src/index.ts
    at Object.getParserServices (/Users/vishalsh/workspace/my-project/node_modules/@typescript-eslint/experimental-utils/dist/eslint-utils/getParserServices.js:23:15)
    at create (/Users/vishalsh/workspace/my-project/node_modules/@typescript-eslint/eslint-plugin/dist/rules/await-thenable.js:41:37)
    at Object.create (/Users/vishalsh/workspace/my-project/node_modules/@typescript-eslint/experimental-utils/dist/eslint-utils/RuleCreator.js:13:24)
    at createRuleListeners (/Users/vishalsh/workspace/my-project/node_modules/eslint/lib/linter/linter.js:761:21)
    at /Users/vishalsh/workspace/my-project/node_modules/eslint/lib/linter/linter.js:933:31
    at Array.forEach (<anonymous>)
    at runRules (/Users/vishalsh/workspace/my-project/node_modules/eslint/lib/linter/linter.js:878:34)
    at Linter._verifyWithoutProcessors (/Users/vishalsh/workspace/my-project/node_modules/eslint/lib/linter/linter.js:1177:31)
    at Linter.verifyWithConfigArray (/Users/vishalsh/workspace/my-project/node_modules/eslint/lib/linter/linter.js:1276:21)
    at Linter.verify (/Users/vishalsh/workspace/my-project/node_modules/eslint/lib/linter/linter.js:1291:25)
error Command failed with exit code 2.
```

The reason for this error is that we are using typescript specific rules in eslint but have not told the ESLint compiler where to look at the configuration for it. So we need to add a typescript config file for ESLint also. You can add `typescript.eslintrc.json` file with the following options:

```
typescript.eslintrc.json

{
  "extends": "./tsconfig.json",
  "compilerOptions": {
    "types": ["jest"]
  },
  "include": [
    "src", "tests"
  ]
}
```

and then tell the ESLint compiler to use this file in `parserOptions` Once all of the above steps are done. the `.eslintrc.json` file will look similar to:

```
.eslintrc.json

{
  "env": {
    "browser": true,
    "es2021": true
  },
  "extends": [
    "eslint:recommended",
    "plugin:@typescript-eslint/recommended",
    "plugin:@typescript-eslint/recommended-requiring-type-checking"
  ],
  "parser": "@typescript-eslint/parser",
  "parserOptions": {
    "ecmaVersion": 12,
    "project": "tsconfig.eslint.json"
  },
  "plugins": ["@typescript-eslint"],
  "rules": {
    "prefer-const": "error",
    "@typescript-eslint/no-unused-vars": "off",
    "@typescript-eslint/no-unused-params": "off"
  },
  "overrides": [
    {
      "files": ["tests/**/*.ts"],
      "env": { "jest": true, "node": true }
    }
  ]
}
```

By default, I have enabled some rules to check now. Also, make sure you add the right path for tests in the overrides for ESLint to work for tests.

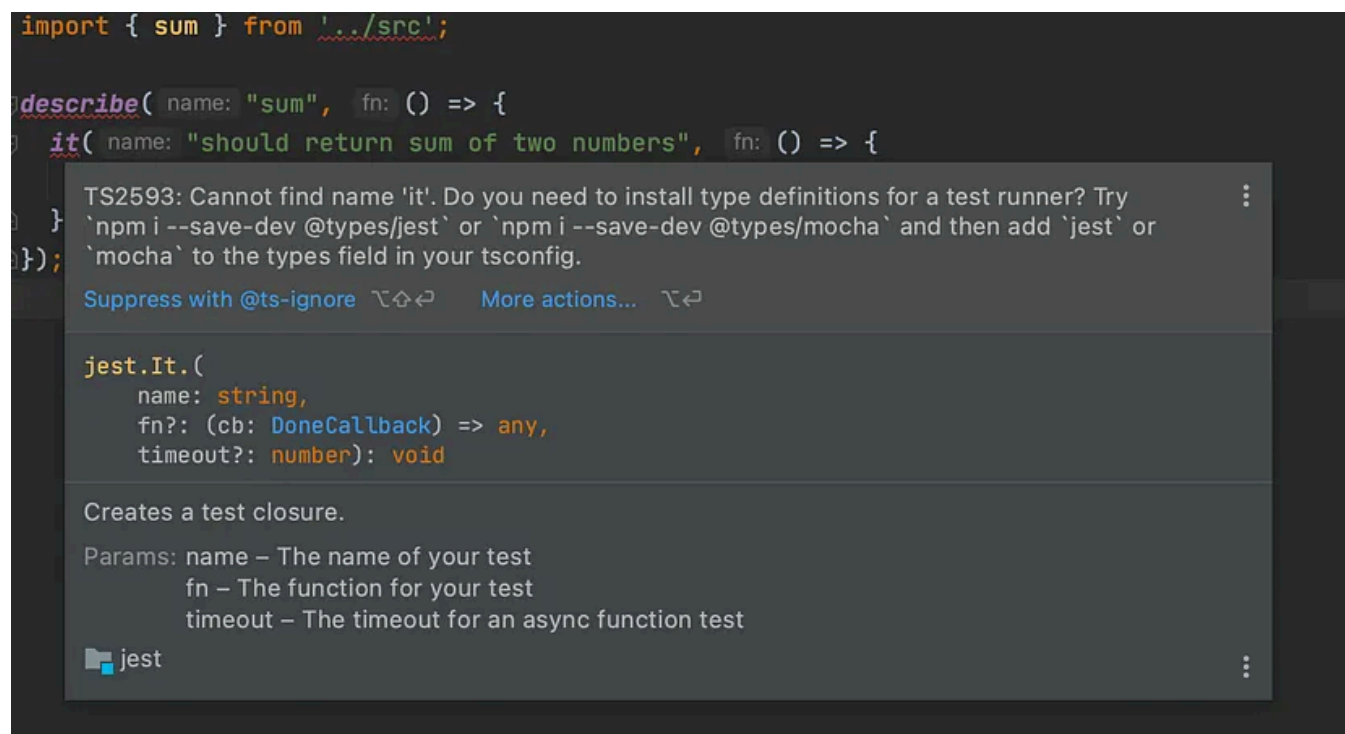
We can try testing lint by executing `npm run lint` now.

Step 5: Setting up Jest

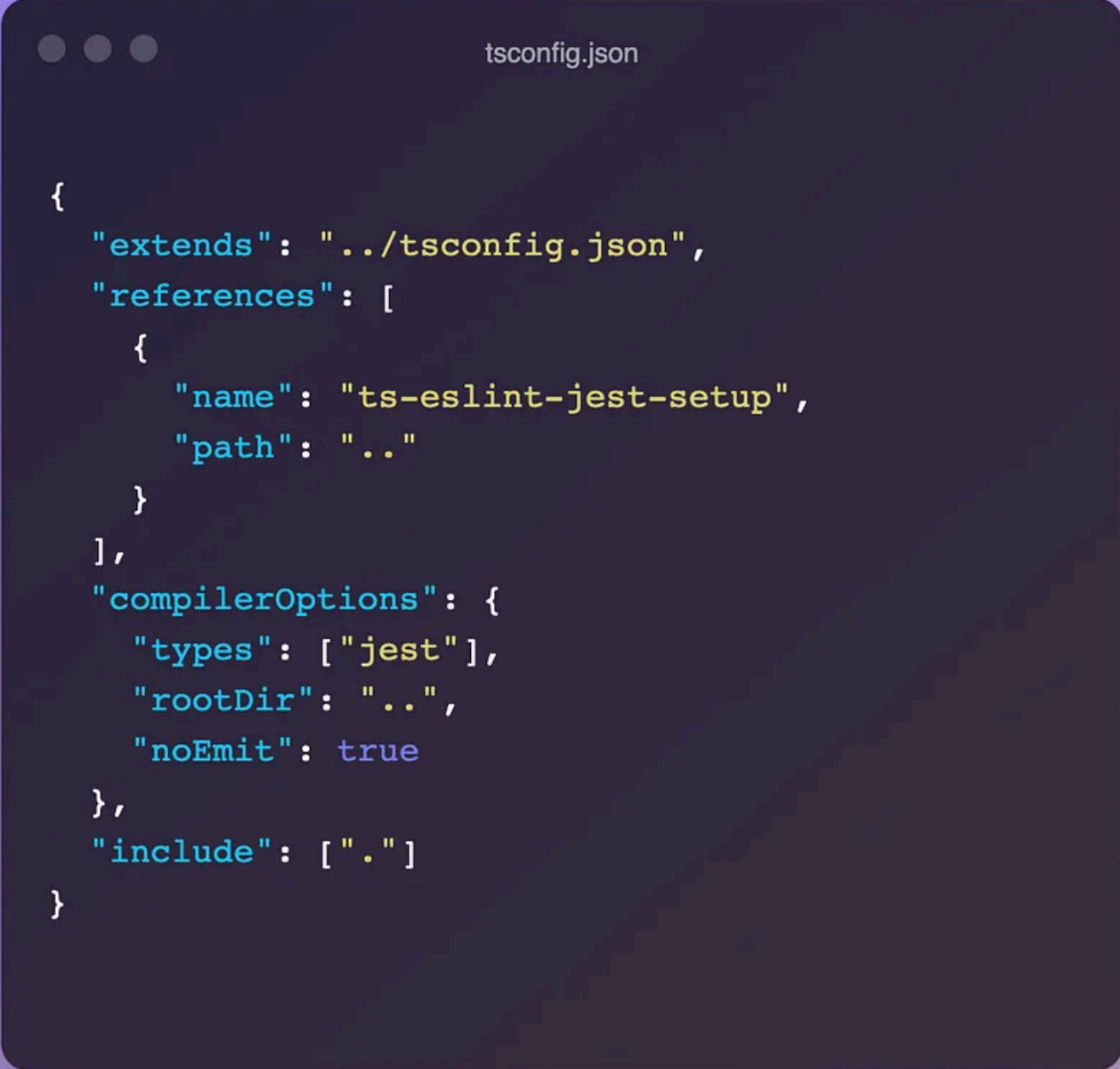
Let's start this step but installing a few dependencies to make jest work with typescript:

```
yarn add -D jest @types/jest @babel/preset-env @babel/preset-  
typescript
```

Now, when we try to add a test for our source code, the IDE might show you the error for the jest functions.



ESLint with typescript enabled does not understand where to find these method types. So our tests also need their .tsconfig file. You can simply create one in the test folder as:



```
{
  "extends": "../tsconfig.json",
  "references": [
    {
      "name": "ts-eslint-jest-setup",
      "path": ".."
    }
  ],
  "compilerOptions": {
    "types": ["jest"],
    "rootDir": "..",
    "noEmit": true
  },
  "include": ["."]
}
```

`references` allow us to create two disconnected sub-projects in the same repo. Make sure to add `"noEmit": true` so that this sub-project does not emit any output. Also to use it, you need to set `"composite": true` in the base `tsconfig` file.

One last thing left is to create a `.babelrc` file for `jest` to understand how to transform the test code and consume it.



`npm run test` should now execute all the tests inside the test folder.

. . .

In the end, you can use the [GitHub boilerplate project](#) to quickly start a new project with the above configuration.

EsLint Config

Typescript

Jest

Frontend Development

EsLintrc



Follow

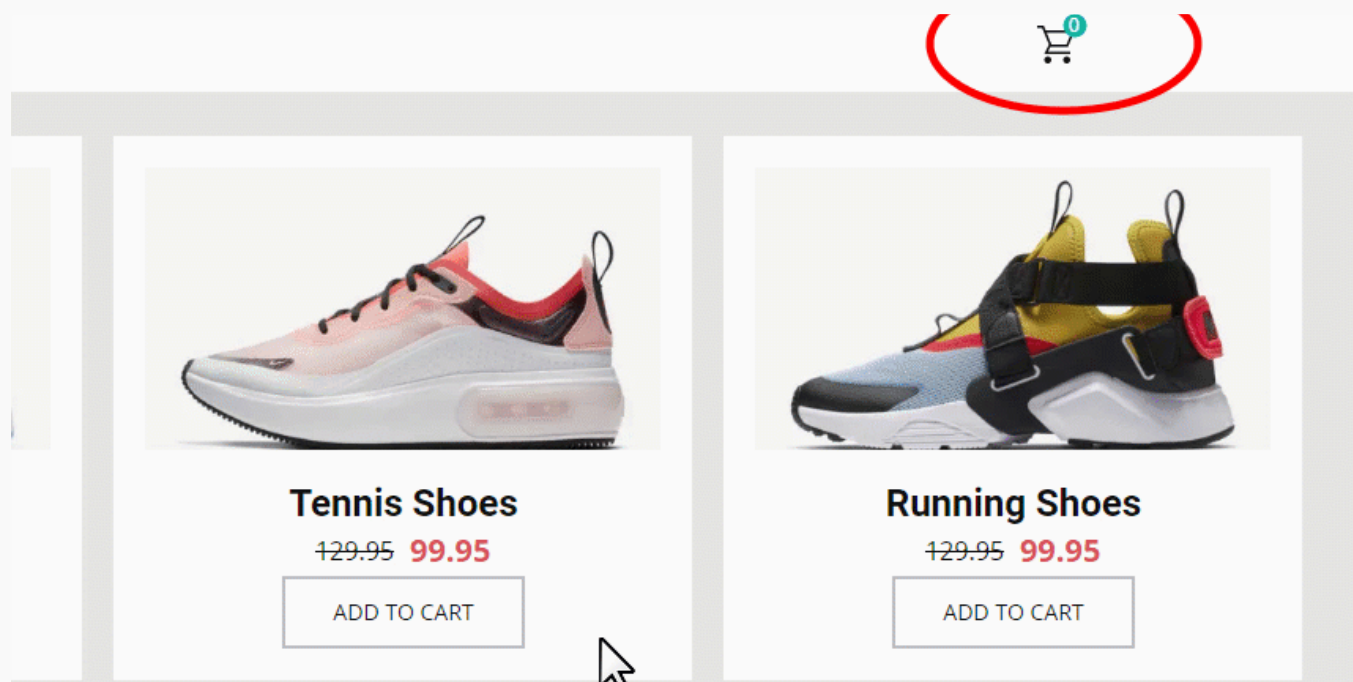


Written by Vishal Sharma

129 Followers

Front End Engineer at /thoughtworks.

More from Vishal Sharma

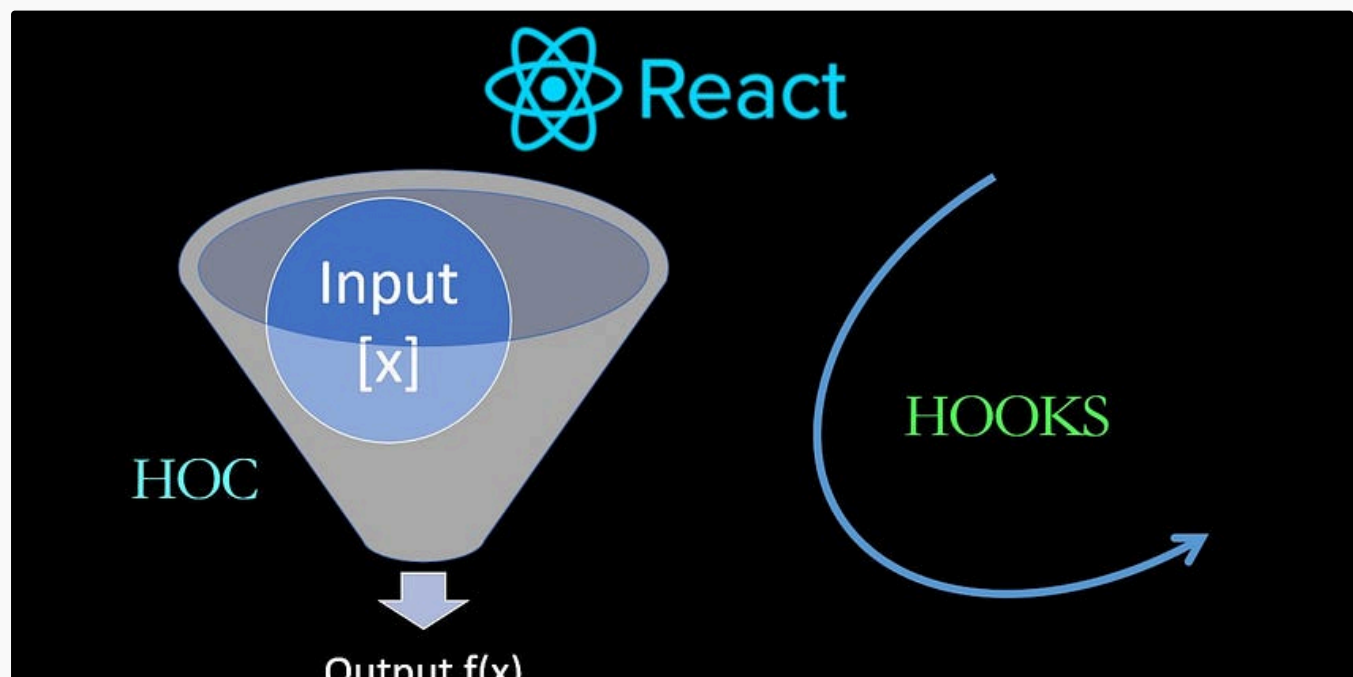


 Vishal Sharma

5 Different Techniques for Cross Micro Frontend Communication

The title itself signifies what we are going to discuss in this article. But before we even start, let's try to understand why cross-micro...

Jun 12, 2022  247  4

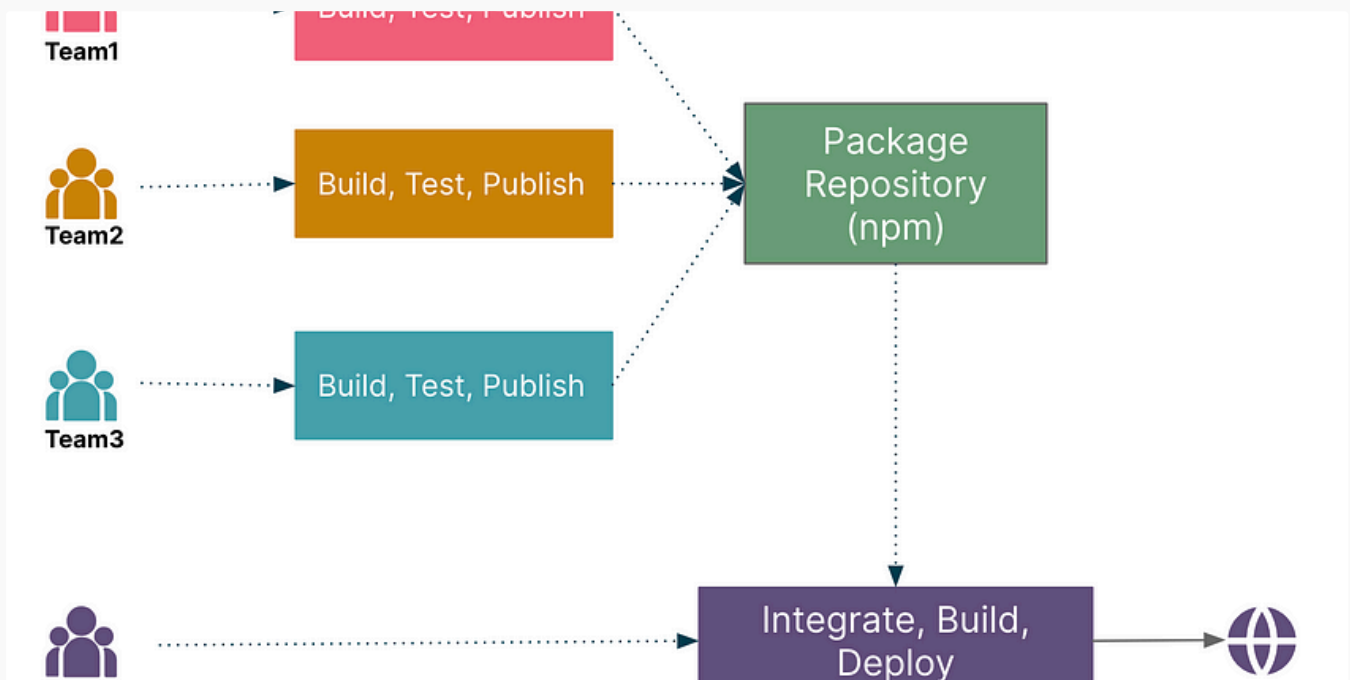


 Vishal Sharma

Following DRY principle for Api Calls in React

In most of the React projects, we have components where we need to fetch the data from an api on mount. React provides us lifecycle...

Nov 5, 2021  296  2



 Vishal Sharma

How to Compose Micro Frontends at Build Time

We have been building npm packages and using those inside the frontend application for eternity now. Build time composition of Micro...

Aug 5, 2022  32  1





Vishal Sharma

SSR and Web Components— A Contentious Couple

Usually, we talk about the technologies that work best together. This article is the exact opposite where I would like to discuss the...

Jul 31 🖱 9



See all from Vishal Sharma

Recommended from Medium



Bret Cameron in JavaScript in Plain English

How to Easily Support ESM and CJS in Your TypeScript Library

A simple example that works for standalone npm libraries and monorepos



May 23



54



-----+ +-----
Common | -> | ES6 Modules
JS | |
-----+ +-----



Rohit Kumar

CommonJS to ES6 Modules to TypeScript: A Practical Guide

It can also be referred as : CommonJS vs ES6 Modules



May 1

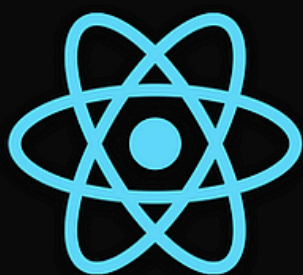


Lists



Natural Language Processing

1754 stories · 1351 saves



ADVANCED REACT

Building Blazing Fast Applications



Imran Farooq in Level Up Coding

React Concepts Put You In The Top 5% Of All React Developers (Ep 2)

So as promised in part 1 of this article's series let's now understand component instances and React elements with the help of the code...



Oct 2



129



1





 Abhishek P N

Setup Jest in a typescript React project

1. Install React testing library dependencies

Apr 23  48  2



Part 02:

Streamline Your Dev Workflow: Husky, Lint-Staged, & Commitlint for Next.js

CONCEPTS

- [Git Hooks](#)
- [Conventional Commits](#)

PACKAGES

- [husky](#)



 Miyushan Rodrigo

Part 02: Streamline Your Dev Workflow: Husky, Lint-Staged, & Commitlint for Next.js

This is the second article of “Recommended Next.js Libraries & Packages” article series. Here I explain how to streamline your developer...



```
my-node-project % npx eslint --init
Also run this command directly using 'npm init @eslint/config'
create-config: v1.1.5

Would you like to use ESLint? · problems
What type of modules does your project use? · esm
Which framework does your project use? · none
Does your project use TypeScript? · javascript
Where does your code run? · browser, node
The config that you've selected requires the following dependencies:
x, globals, @eslint/js
Do you like to install them now? · No / Yes
Which package manager do you want to use? · npm
Installing...
```



Sindhu

Setting Up ESLint and Prettier in a Node.js Project

Linting and code formatting are crucial for maintaining a consistent and error-free codebase. ESLint helps identify and fix problems in...

[See more recommendations](#)